AI 시대, 개발자는 무엇으로 남는가

주니어 개발자의 관점에서

오늘 함께 생각해볼 질문

"AI가 코드를 다 짜준다면, 우리는 무엇을 해야 할까요?"

- 최근 한 달 동안 **AI에게 맡긴 일**은 무엇인가요?
- 끝까지 **직접 한 일**은 무엇인가요?

> **막연한 불안 대신, 내 일에서 실제로 무엇이 바뀌었는지부터 보겠습니다.**

지금 무슨 일이 일어나고 있을까요

- **주니어 채용은 줄고**, 경력자 채용은 상대적으로 유지되고 있습니다
- 다른 직군보다 개발이 가장 먼저 AI의 영향을 받은 이유
  - 학습할 코드 데이터가 많고
  - 실행해보면 맞는지 바로 검증할 수 있고
  - 입력도 출력도 전부 텍스트이기 때문입니다

> **개발자는 AI에게 가장 먼저 대체된 게 아닌, 가장 먼저 적응을 시작한 직업입니다.**

딥다이브, 아직 유효할까요?

가치가 떨어진 공부	가치가 오른 공부
단순 언어 문법 암기	동작 원리 이해
라이브러리 사용법	트레이드오프 판단
단순 구현 연습	깊은 디버깅

> **"어떻게 쓰는가"보다 "왜 이렇게 동작하는가"를 파야 합니다.**

그런데 AI가 그것마저 잘한다면?

AI는 판단 영역까지 빠르게 들어오고 있습니다.
그래도 사람에게 남는 것이 있습니다.

- **맥락** : 이 기능이 비즈니스에서 왜 중요한지
- **문제 정의** : 무엇을 물어야 하는지
- **책임** : 배포는 결국 사람이 합니다
- **검증** : AI가 그럴듯하게 틀릴 때 이를 알아채는 능력

> **AI가 발전할수록, 틀린 답도 더 그럴듯해집니다.**

기술적 깊이가 이동하고 있습니다

이전	이후
직접 고려해서 **구현하는** 능력	**판단하고, 업무를 나누고, 맡기는** 능력

> **깊이가 사라지는 게 아니라, 깊이를 쓰는 방식이 바뀌는 겁니다.**

기획과 책임의 비중이 커집니다

PM의 기획이 아니라, **기술을 알기 때문에 가능한 기술적 기획**입니다.

- "SSR로 가면 SEO와 초기 로딩은 좋아지지만 서버 비용이 늘어난다."
  - 하지만 이를 감안해도, 비즈니스 임팩트가 크기 때문에 채택한다.
- "이 작업은 AI에게 맡기고, 이 부분은 직접 확인하자"
- "지금은 돌아가지만 6개월 뒤에 데이터가 많아지면 문제가 되겠다"

이런 흐름에서 나온 말이 **프로덕트 엔지니어**입니다.

> **시니어가 하던 일을 더 빨리 요구받는 시대입니다.**

주의해야 할 4가지 함정과 대안

함정	무엇이 문제인가	이렇게 길러봅시다
경험이 쌓이지 않는다	판단력은 구현 경험에서 나오는데, AI가 구현을 가져갑니다	처음 배우는 개념과 핵심 로직은 AI 없이 먼저 짜보기
이해 부채가 쌓인다	돌아가니까 머지하다 보면, 아무도 모르는 코드가 쌓입니다	머지 전에 "AI 없이 동료에게 설명할 수 있나?" 스스로 묻기
검증이 형식적으로 변한다	열 번 맞으면 열한 번째는 대충 보게 됩니다	리뷰할 때 "맞나?" 대신 "어디서 깨질까?"를 찾기
생각까지 넘겨버린다	AI의 첫 답을 채택하면 판단자가 아니라 승인자가 됩니다	내 답을 먼저 정하고, 대안 2~3개와 비교하기

> **AI가 편해질수록, 내가 이해하는 범위가 조용히 줄어듭니다.**

회사는 속도를 원하는데요?

- AI로 빠르게 만드는 건 이제 **누구나** 할 수 있습니다
- 차이는 **깊은 이해를 바탕으로 빠르게 만들 수 있는지**에서 납니다
- 깊이는 평소엔 안 보이다가, 장애가 나거나 서비스가 커질 때 드러납니다
- **업무와 연결된 깊이**를 확장하면서 이런 능력을 기를 수 있습니다

> **깊이와 속도는 대립하지 않습니다. 깊이가 있어야 속도를 안전하게 낼 수 있습니다.**

더 오래가는 무기: 도메인과 넓이

도메인 지식 : 규제, 표준, 운영에서 생기는 예외

T자형 개발자

- 세로축: 주력 분야는 원리까지 깊게
- 가로축: API, 인증, 캐싱, 인프라는 판단할 수 있을 만큼

> **"프론트엔드 개발자"에서 "프론트엔드에 강한 제품 개발자"로.**

주니어는 어떻게 시니어가 될까요?

AI가 단순한 일을 가져가면서 **성장 사다리의 아래 칸이 비어버렸습니다.**

개인이 할 수 있는 것

- AI를 정답지가 아니라 선생님으로 쓰기 : "왜 이렇게 짰어?"
- 코드를 짜기 전에 내 설계를 먼저 적어보기
- 리뷰를 받는 것뿐 아니라, 리뷰를 하면서 성장하기
- 코드 밖의 맥락, 사용자와 비즈니스를 일찍 이해하기

팀이 해줘야 하는 것

- 성장용 일을 의도적으로 남겨두기
- 결과보다 "왜 이렇게 선택했어?"를 묻기

> **예전엔 경험이 저절로 쌓였지만, 이제는 의도하지 않으면 쌓이지 않습니다.**

다른 직업들은 안전할까요?

화면 앞에서 정보를 다루는 지식 노동은 **모두** 영향을 받고 있습니다.
번역, 콘텐츠 작성, 고객 상담, 디자인, 법률 문서 검토…

예술은 어떨까요?

- 기능적인 창작물은 대체되고 있지만
- 사람들은 작품만큼 **사람의 이야기와 존재**를 소비합니다
- 사진이 나와도 회화는 사라지지 않고, 새로운 방향으로 나아갔습니다

> **만드는 행위는 AI가 가져가도, 판단하고 이름을 거는 일은 사람에게 남습니다.**

정리하자면

- 원리는 **깊게**, 구현은 **유연하게**
- 경험과 실패를 **의도적으로** 만들고, 반드시 돌아보기
- **판단하고 책임지는 사람**으로 성장하기

> **AI를 가장 잘 쓰는 사람이 되, AI 없이도 판단할 수 있는 사람으로 남읍시다.**

마지막으로

- 이 방향이 정답인지는 **저도 모르겠습니다**
- 다만 어떤 미래가 오더라도, **깊이 이해하고 스스로 판단해본 경험**은 남을 거라고 생각합니다
- 정답을 찾기보다, 변화를 계속 지켜보며 **함께 방향을 고쳐 나갔으면** 합니다